

All information in this document is also maintained on the project GitHub Wiki (Home and Person-Hours Estimation pages). This PDF is a repository copy for grading on the master branch.

Project Overview

A single-player Minesweeper game on a 10×10 grid. Players uncover cells to reveal numbers indicating adjacent mines, flag suspected mines, and clear the board without detonating a mine. Mine count is user-specified (10–20). Written in C++ with a Qt GUI; CMake manages the build.

Team

Name	Role
Will Godderz	Project Manager
Jaydee Brown	Frontend Developer
Lea Sulthana Syed	Quality Testing (UI & Input)
Aidan Bergeman	Version Control & Code Quality
Zackary Rhodes	Backend Developer
Ethan Gao	Backend Developer
Didier Carrillo	Technical Lead

Technology Stack

- Language: C++ (C++17)
- Front end / GUI: Qt6 (Widgets)
- Build system: CMake (3.21+)

Meeting Notes

Meeting 1 — Tuesday, Sep 2, 2026

Attendees: Full team

Decisions: collected contact info; team meets Wed 1:00–2:00 PM and TA Fri 9:00–10:00 AM; communicate via Slack; create a GitHub repo for all code and docs; develop in C++ with a Qt UI.

Action items: set up Slack workspace; create GitHub repo and add collaborators; fill in contact info; set up the Qt/C++ skeleton.

Meeting 2 — Friday, Sep 11, 2026

Attendees: Full team

Decisions: use the Use Case Points method (cross-checked with a bottom-up estimate); assign roles (Jaydee frontend; Ethan and Zackary backend; Will PM; Lea QA; Didier tech lead; Aidan version control); pipeline is Input Handler → (coordinate, action) array → backend stores data → logic handler computes the outcome → frontend displays.

Action items: create the frontend interface; enable user input intake; implement game logic; implement board storage.

Meeting 3 — Friday, Sep 18, 2026 (9:00–10:00 AM, Eaton 3002)

Attendees: Full team

Items completed: frontend interface (Jaydee); user input intake (Jaydee, Will); board information storage (Zackary); functional game logic (Ethan, Will); working demo and program comments (Lea, Will); project and data-flow diagrams (Didier).

Decisions: add better representation of how data flows to the diagrams. Action items: improve and test build instructions.

Actual Person-Hours (Day-by-Day)

Hours logged per member, excluding EECS 581 lecture time. Days with no recorded hours are omitted.

Date	Will	Jaydee	Lea	Aidan	Zackary	Ethan	Didier
9/2	1	1	1	1	1	1	1
9/9			1				
9/12		4					
9/13		2				1.5	
9/14				1			1
9/15						0.5	3
9/16	1	1	1	1	1	1	3
9/17	4.5	1	3				
9/18	1	1	1	1	1	1	1
9/19				3			
9/20							3
Total	7.5	10	7	7	3	5	12

Team total: 51.5 person-hours (individual work + meetings).

Person-Hours Estimation (Use Case Points)

Estimated with the Use Case Points (UCP) method from lecture, sanity-checked with a bottom-up task estimate.

One actor — the Player via the Qt GUI (Complex = 3), so UAW = 3. Use-case weights: Simple = 5, Average = 10, Complex = 15.

Use case	Complexity	Weight
Start / configure a new game	Average	10
Uncover a cell (recursive flood-fill)	Complex	15
Flag / unflag a cell	Simple	5
Evaluate game outcome (win/loss)	Average	10
View board & status	Simple	5
UUCW total		45

Step	Result
Unadjusted UCP (UUCW 45 + UAW 3)	48
Technical Complexity Factor (TCF)	0.835
Environmental Complexity Factor (ECF)	0.92
Adjusted UCP (48 × 0.835 × 0.92)	≈ 36.9
Adopted effort	≈ 130 hrs

The textbook 20 hrs/UCP rate predicts ~738 hrs, which is unrealistic for a small game, so we adopt a bottom-up estimate of ~130 person-hours (~3.5 hrs/UCP), using UCP as a cross-check. Assumptions: fixed 10×10 grid; mines 10–20; single GUI player; no networking, accounts, or timer. Actual hours are logged day-by-day and compared to this estimate.